

CSE 331: Computer Security Fundamentals Spring 2026

Side Channels

Amir Rahmati
Xinrui Yu



Stony Brook Institute
at Anhui University

安徽大學紐約石溪學院

Side Channel Attacks

- Attacks based on information that can be gleaned from the physical implementation of a system, rather than breaking its theoretical properties
 - Initially most commonly discussed in the context of cryptosystems
 - But also prevalent in many contexts

Examples on Cryptosystems

- Timing attacks
- Power analysis
- Thermal analysis
- Electromagnetic channel
- Good overview:
http://www.nicolascourtois.com/papers/sc/sidech_attacks.pdf

If you do something different for secret key bits 1 vs. 0, attacker can learn something...

TENEX Timing Attacks

- 1970s: TENEX operating system compares user-supplied string against secret password one character at a time, stopping at first difference:
 - AAAAAA vs. SECRET: stop at 1.
 - SAAAAA vs. SECRET: stop at 2.
 - SEAAAA vs. SECRET: stop at 3.
- Attacker sees comparison time, deduces position of difference.
- A few hundred tries reveal secret password.

```
for (i = 0; i < 16; ++i)
    if (x[i] != y[i])
        return 0;
return 1;
```

```
int result = 0;
for (i = 0; i < 16; ++i)
    result |= x[i] ^ y[i];
return result == 0;
```

Notice that the language makes the wrong thing simple and the right thing complex.

Why Is This Called a "Side-Channel"?

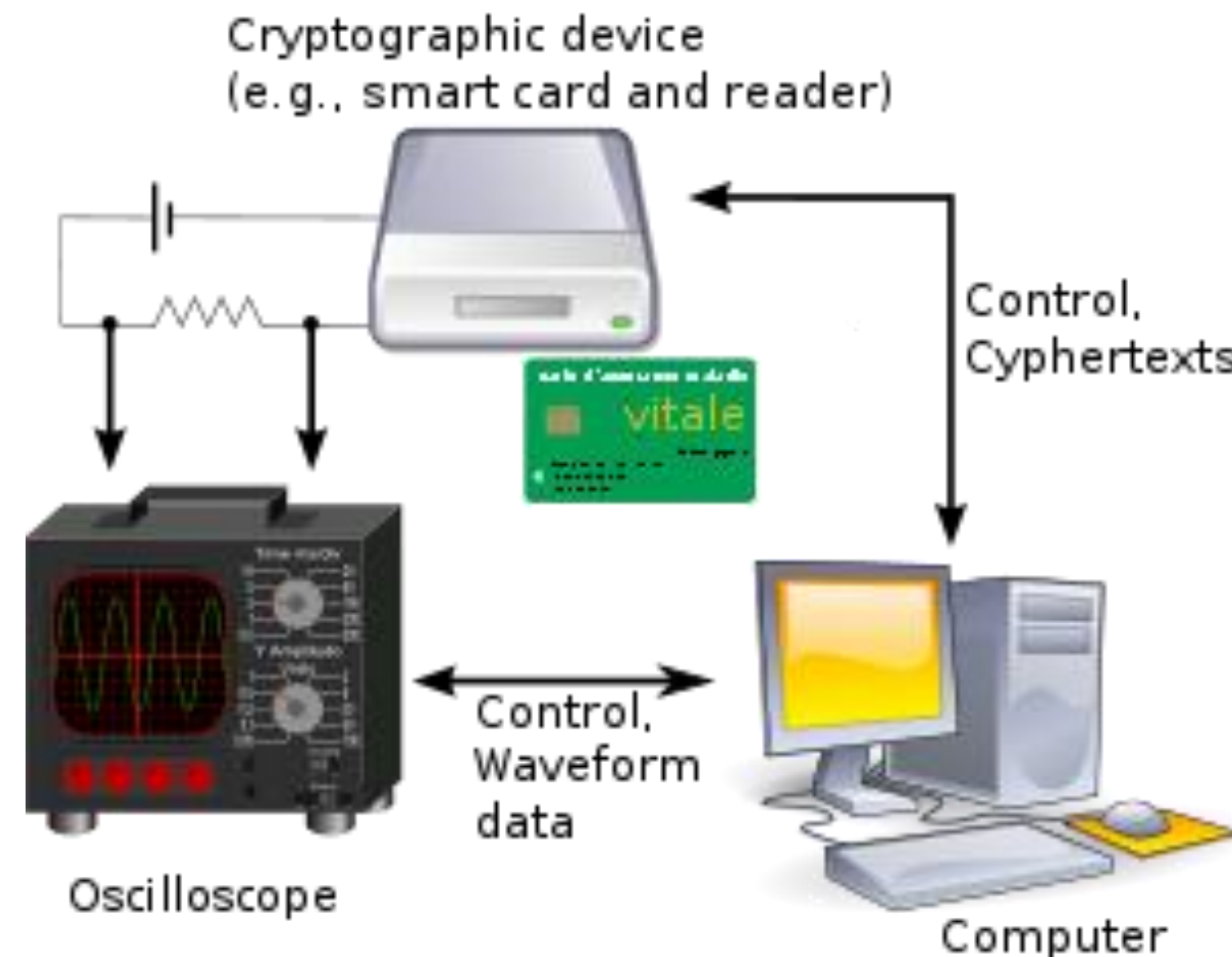
- **The Main Channel:** The intentional input and output (you send a guess, the card sends back a digital "Yes" or "No"). The "No" tells you nothing about *why* it failed.
- **The Side Channel:** The unintentional physical byproduct of the computation (time elapsed, power consumed, electromagnetic radiation emitted).

Example Timing Attacks

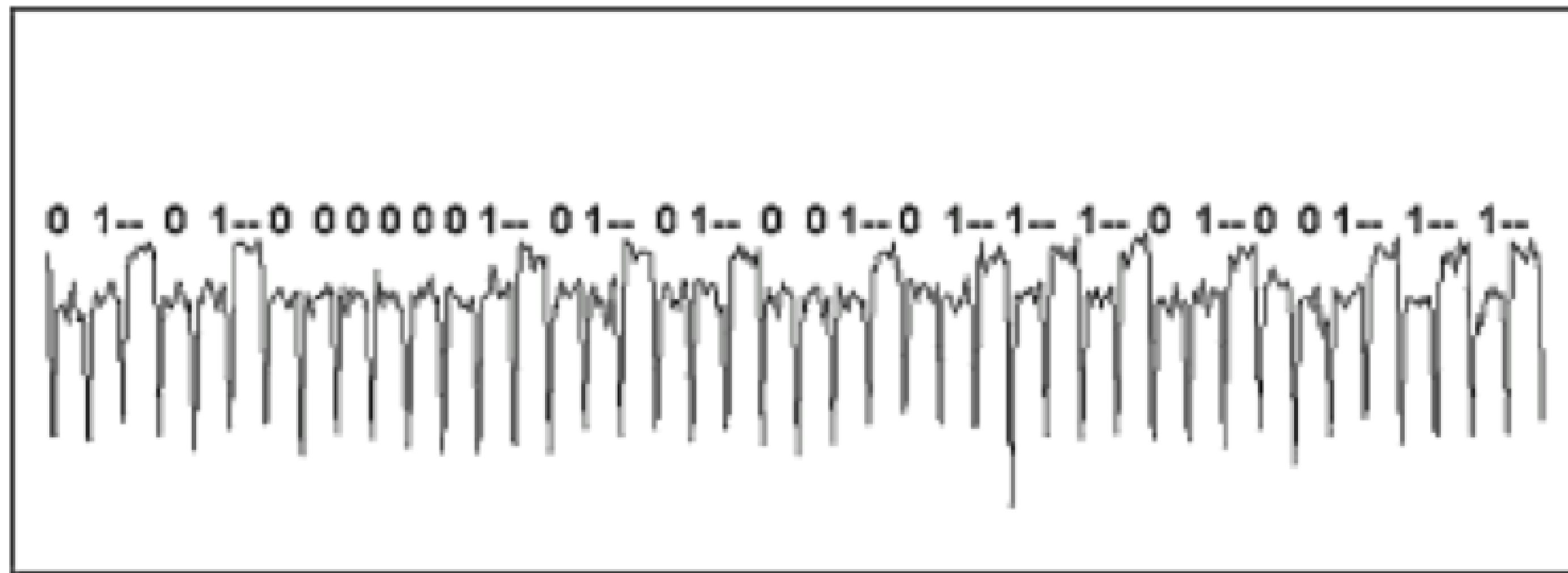
- **RSA:** Leverage key-dependent timings of modular exponentiations
 - <https://www.rambus.com/timing-attacks-on-implementations-of-diffie-hellman-rsa-dss-and-other-systems/>
 - <http://crypto.stanford.edu/~dabo/papers/ssl-timing.pdf>
- **Block Ciphers:** Leverage key-dependent cache hits/misses

Power Analysis

- **Simple power analysis:** Directly read off bits from powerline traces
- **Differential power analysis:** Look for statistical differences in power traces, based on guesses of a key bit

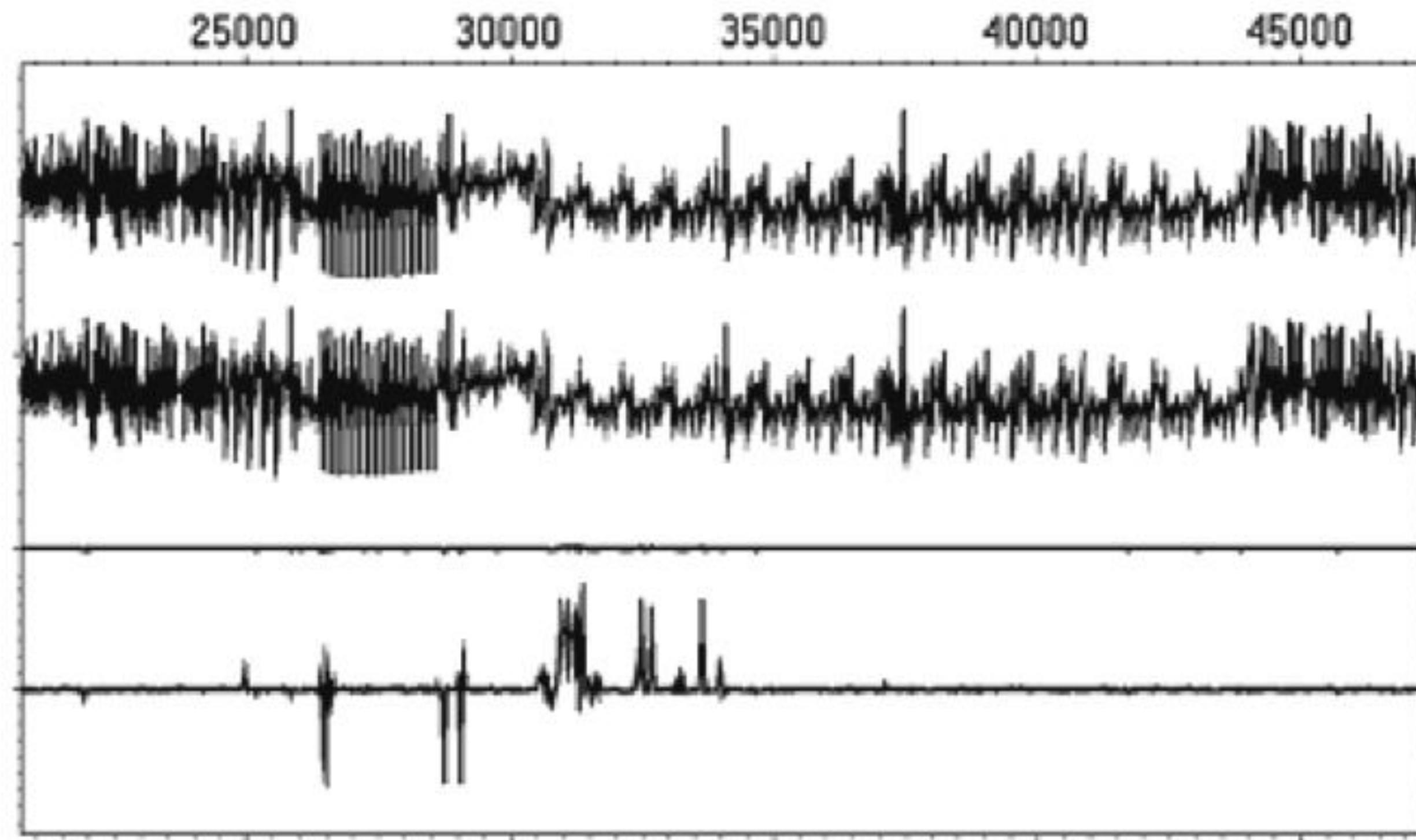


Simple Power Analysis

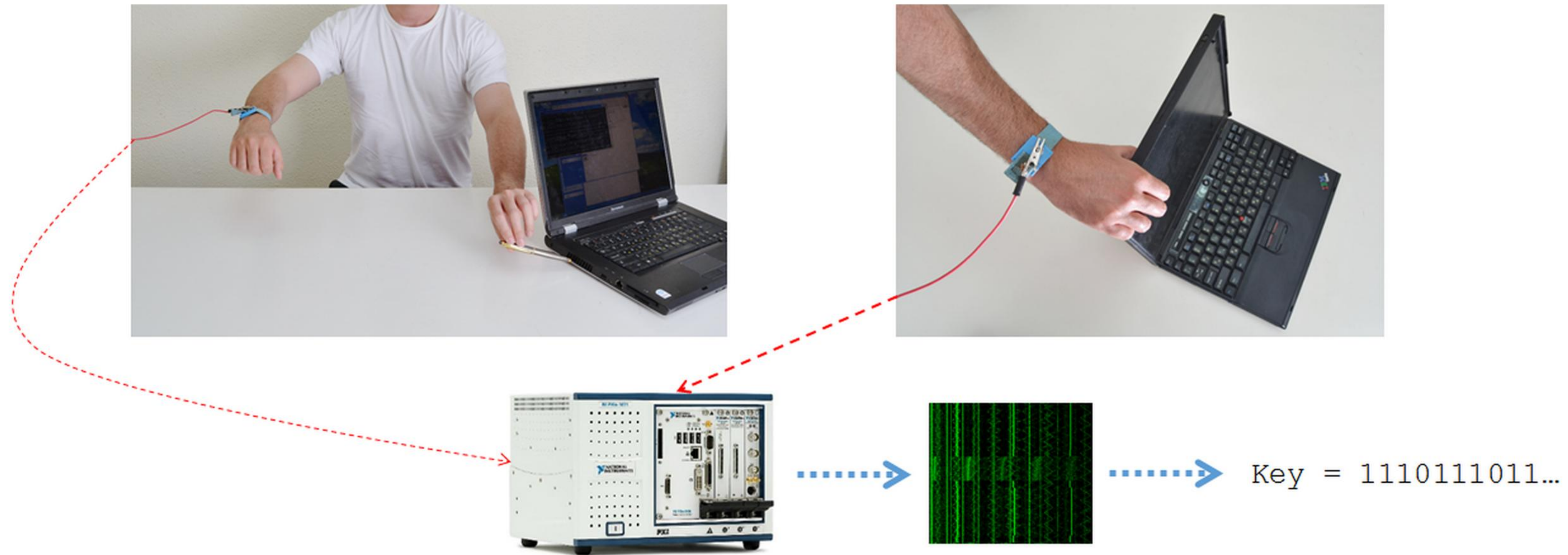


- Distinguish 1s from 0s
- Distinguish addition from multiplication
- Ratios of 0's and 1's

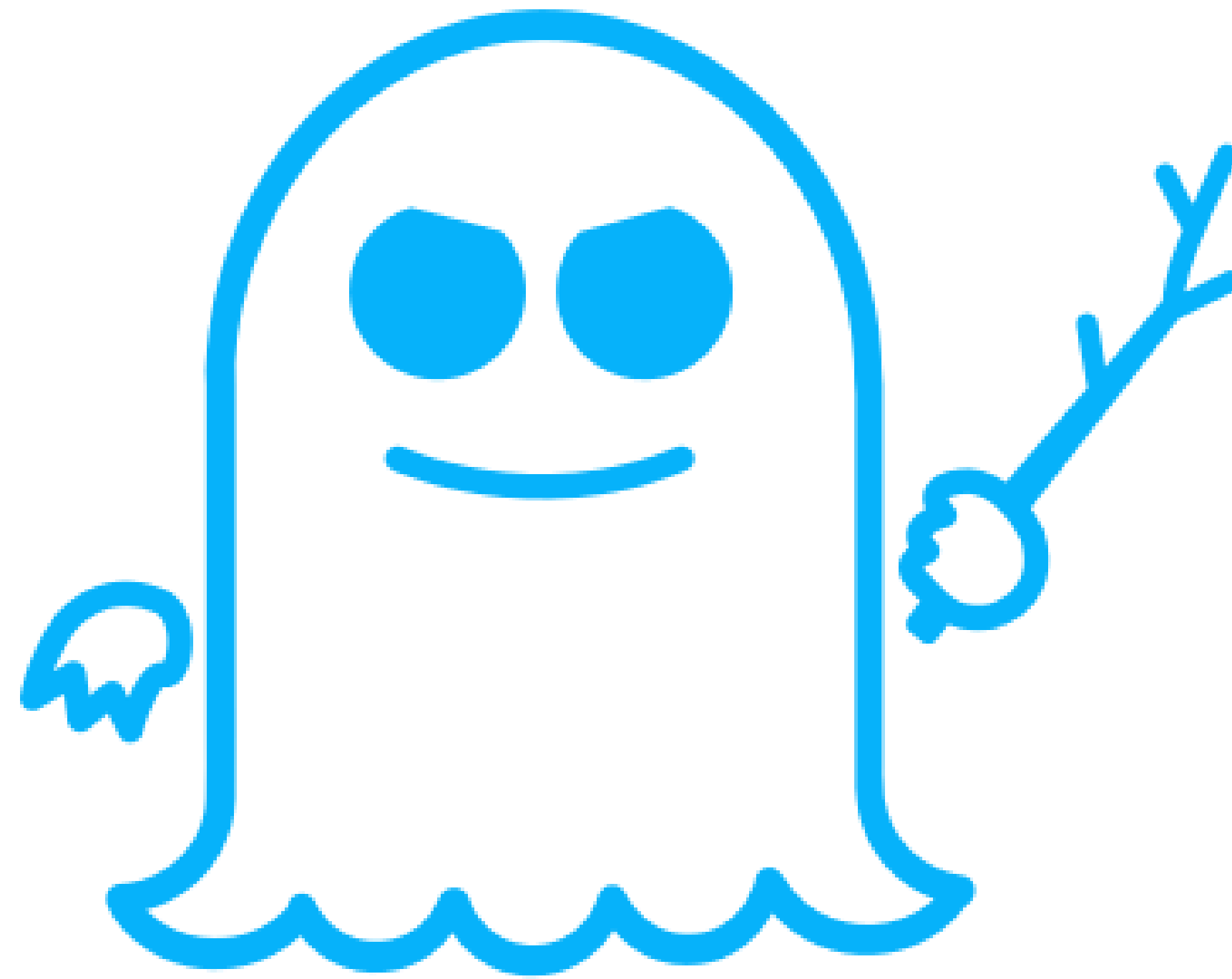
Differential Power Analysis



Key Extraction via Electric Potential



The attackers discovered that by touching a plain wire to an exposed metal part of the laptop chassis, they can determine the bits of a secret key. A 0 bit results in a short, low-power calculation. A 1 bit results in a long, high-power calculation.



SPECTRE

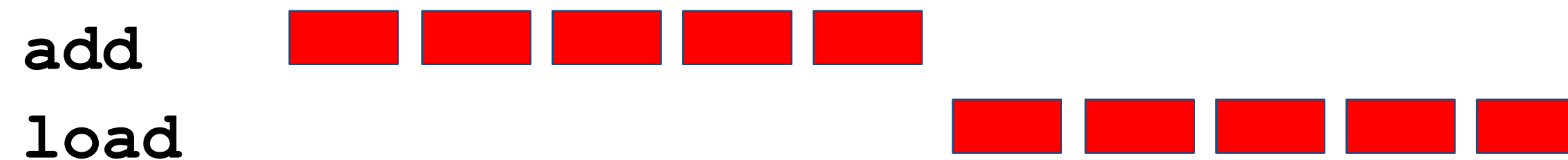
Spectre

Exploit **(1) speculative execution** and **(2) cache timing information** to **extract private information from the same process**

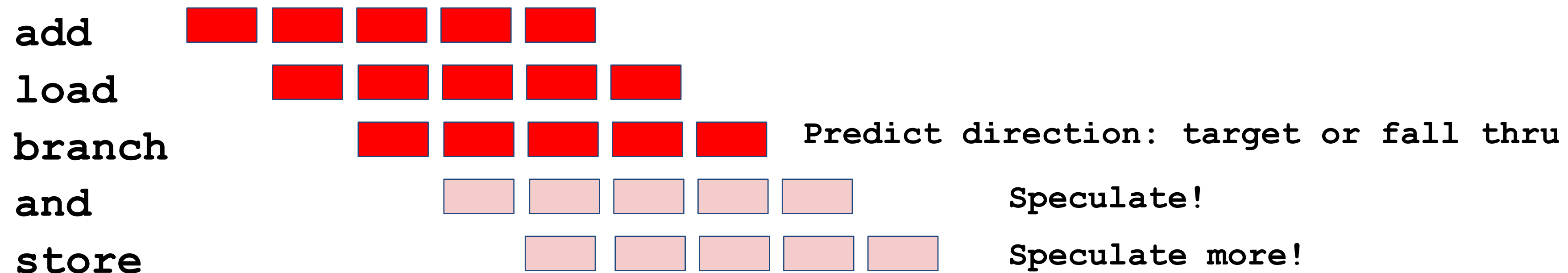
- Example: JavaScript running in web page

Instruction Speculation

Many steps (cycles) to execute one instruction; time flows left to right →



Go Faster! Pipelining, branch prediction, & instruction speculation



If speculation correct: Commit **architectural** changes of **and** (**register**) & **store** (**memory**) go fast!

If mis-speculate: Abort **architectural** changes (**registers, memory**); go in other branch direction

Speculation affects the cache!

Cache Timing Attacks

- **Basic idea:** Manipulate cache to a known state, wait for victim activity, then examine what has changed.
- Example: “Flush + Reload”
 1. Flush relevant addresses from cache
 2. Wait for victim (here: execute Spectre attack)
 3. Reload relevant addresses, time accesses

If timing was fast, victim accessed relevant address; if timing was slow, victim did not.

Spectre Threat Model

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

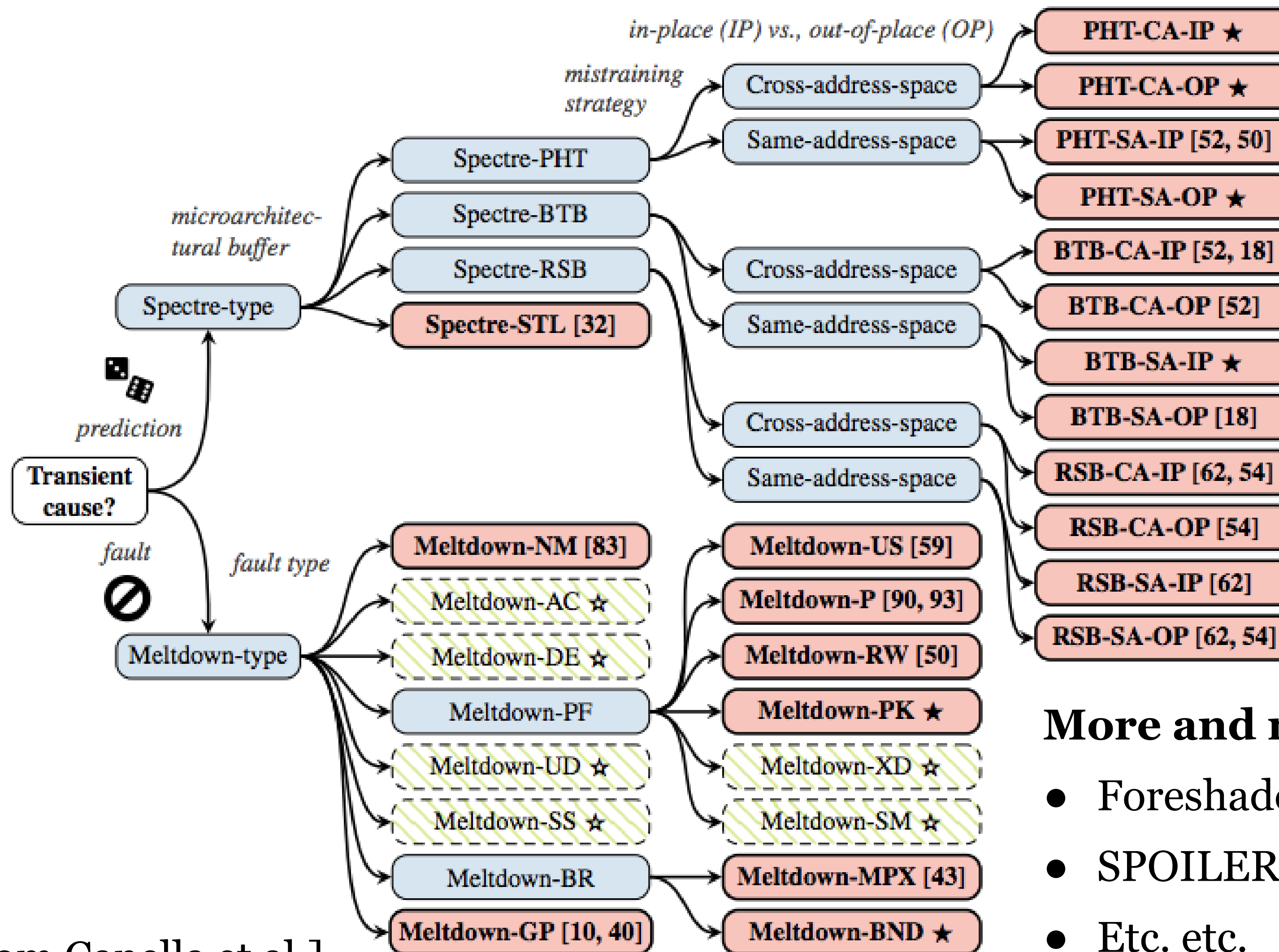
- Suppose that:
 - Adversary can run code, in the same process.
 - Adversary can control the value x.
 - Adversary has access to array2.
 - Adversary cannot just read arbitrary memory in the process.
 - There is some secret value, elsewhere in the process, that the adversary would like to learn.
 - Specifically: **Attacker's goal is to read secret memory at address array1+x**

Spectre Attach Sketch

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

1. Train branch predictor to follow one branch of conditional.
2. After training, make the followed branch access information that the code should *not* be allowed to access (`array1[x]`).
3. The secret information will affect which part of `array2` is loaded into the cache.
4. After the hardware determines that the branch was incorrectly executed, the logic of the program will be rolled back *but* the cache will still be impacted.
5. Time reads to cache (e.g., Flush+Reload) for accesses to `array2`, to see which cache lines are read more efficiently.

It's A Party



More and more:

- Foreshadow – attacks SGX
- SPOILER – mem dependence
- Etc. etc.

[From Canella et al.]

Hertzbleed Attack

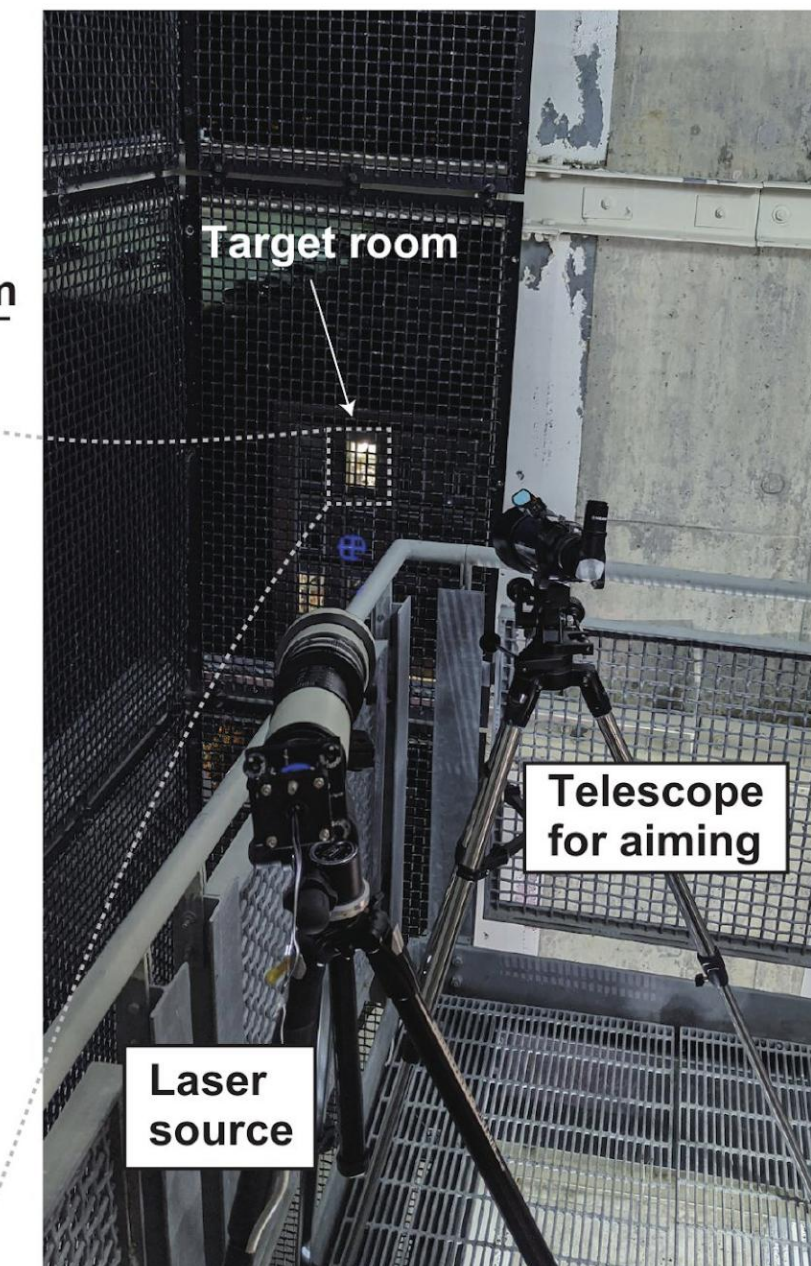
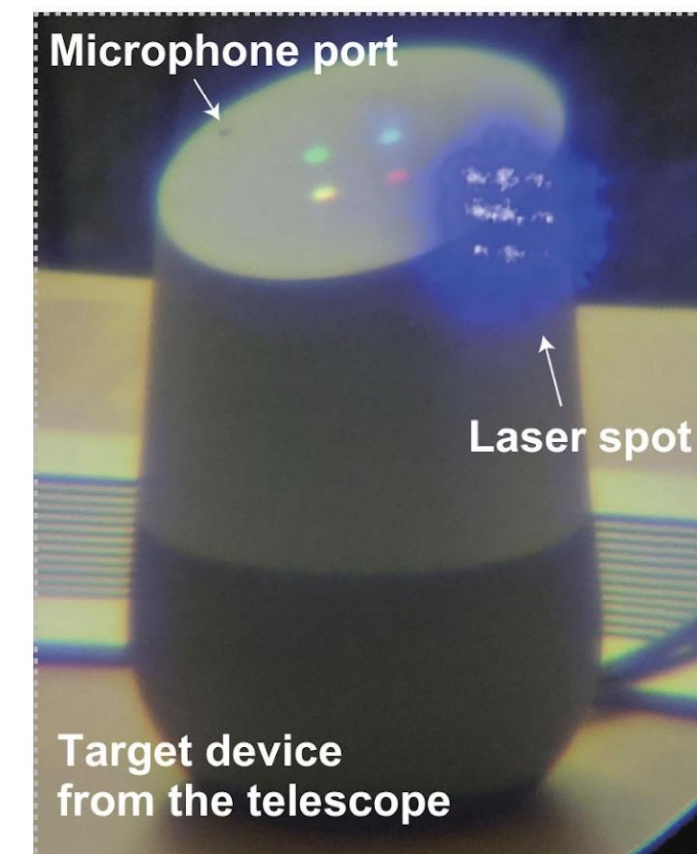
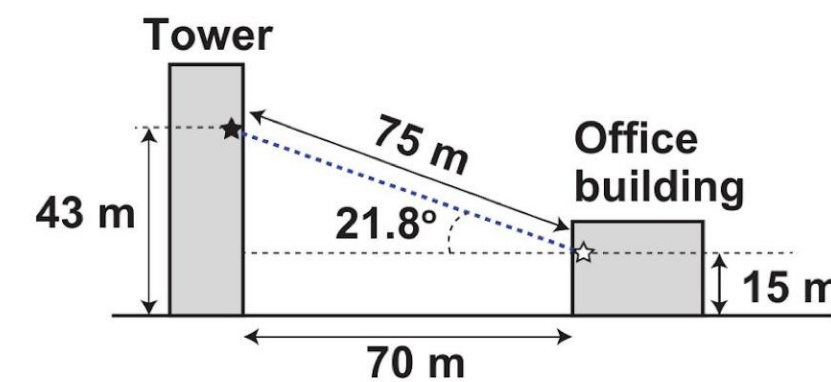
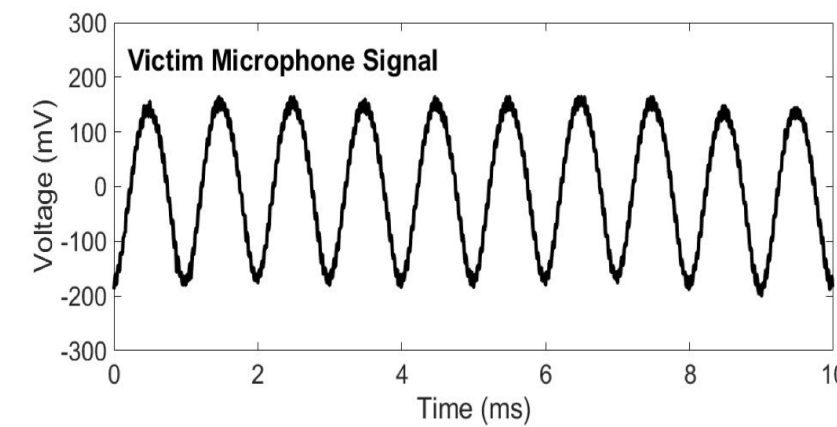
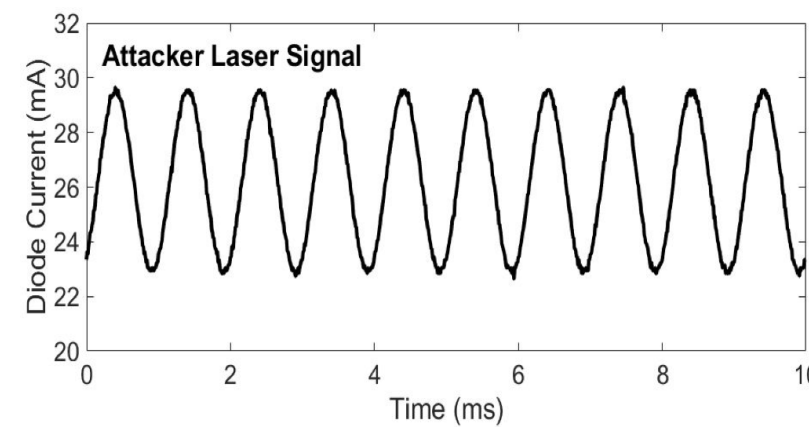
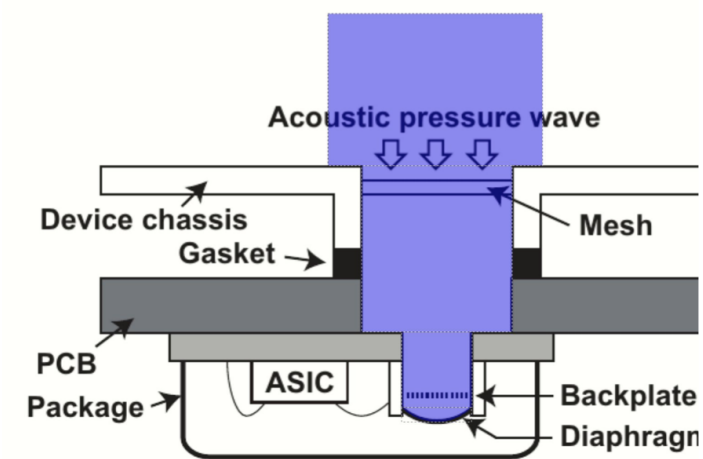


Other “Fun” Examples...

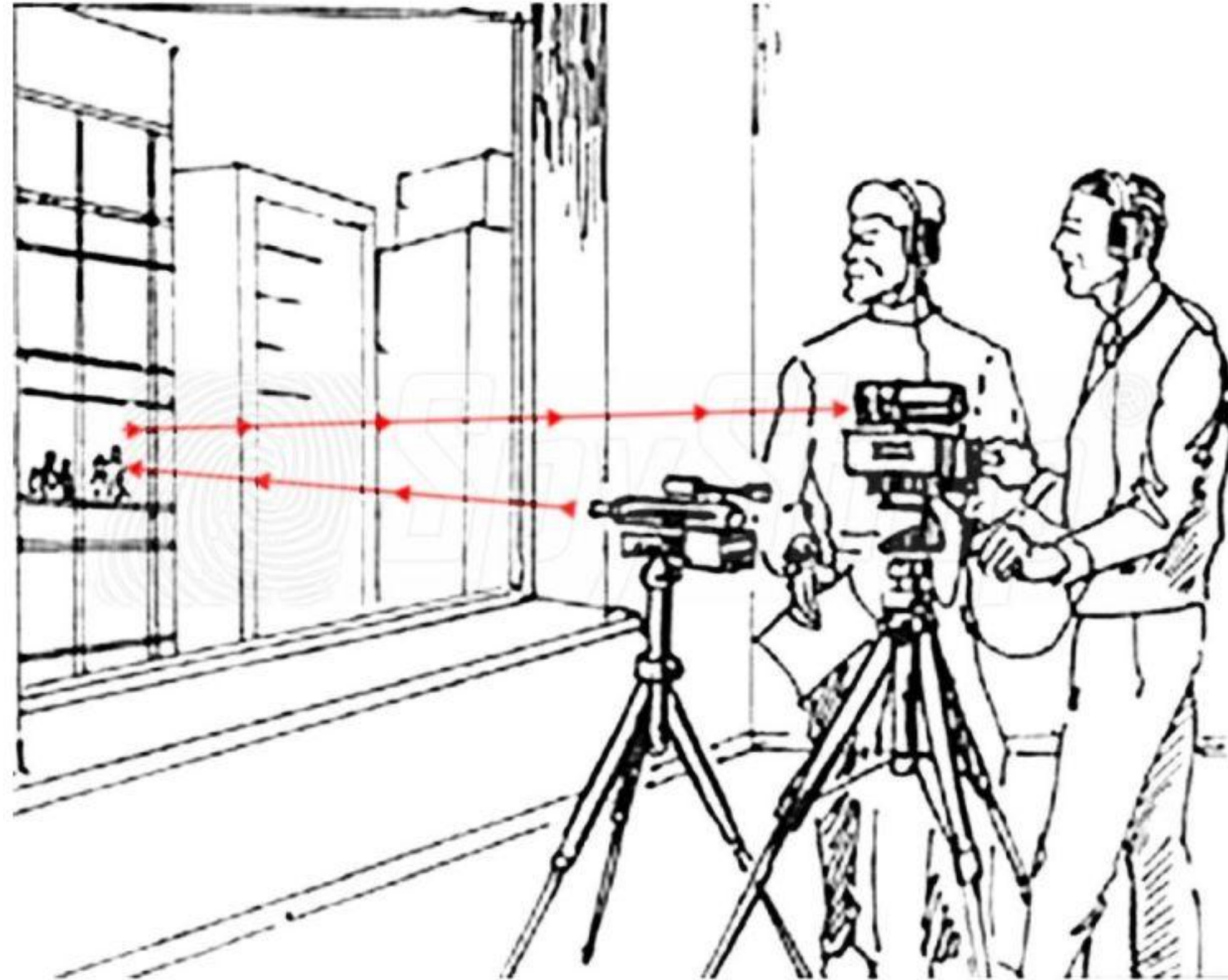
Light Commands: Laser-Based Audio Injection Attacks on Voice-Controllable Systems

Key ideas

1. Amplitude modulated light generates a voltage signal on microphone output
2. Higher amplitude light == higher amplitude voltage
3. Very little distortion



Laser Microphones



Acoustic cryptanalysis

Keyboard Acoustic Emanations

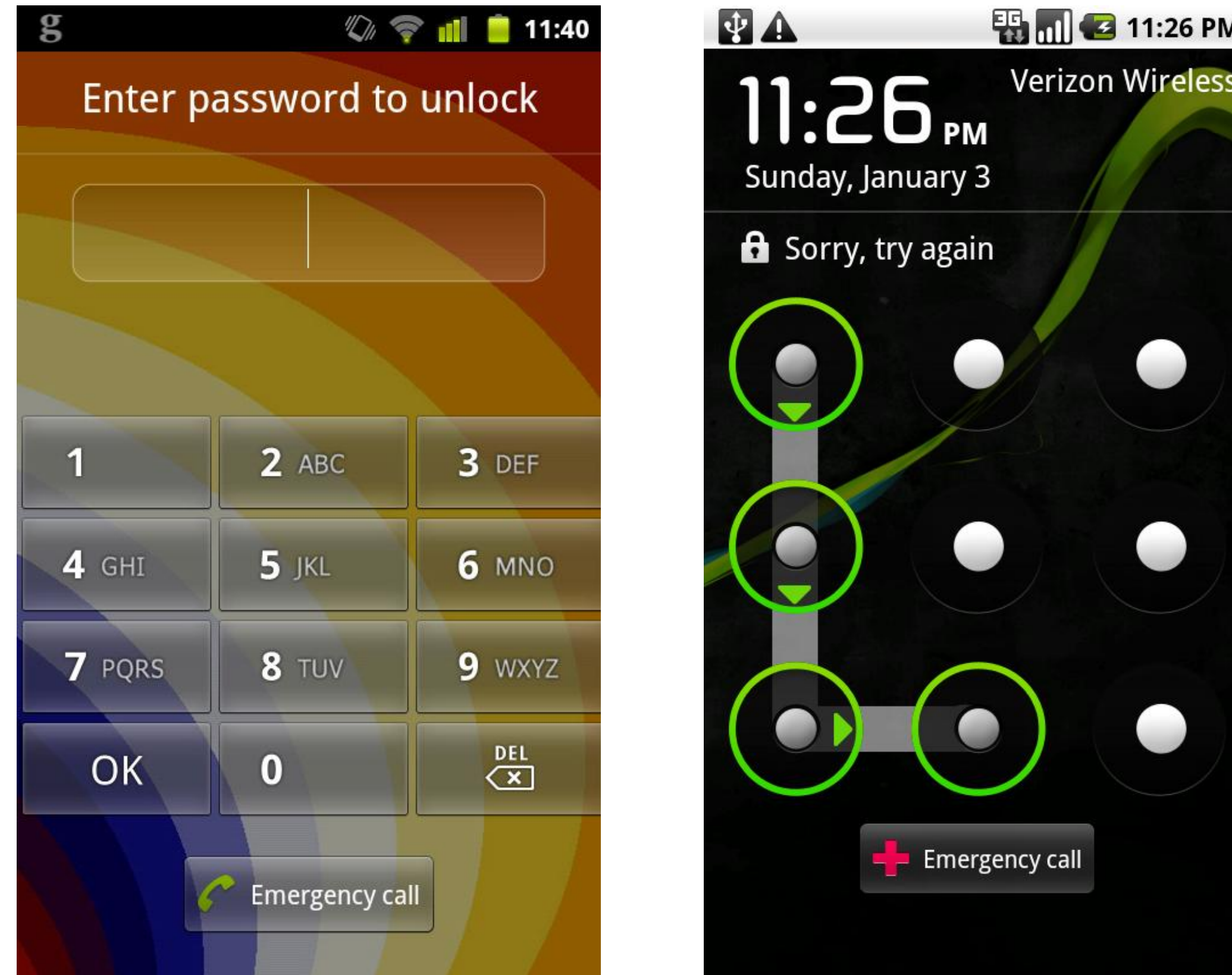
Dmitri Asonov Rakesh Agrawal

IBM Almaden Research Center

650 Harry Road, San Jose, CA 95120, USA

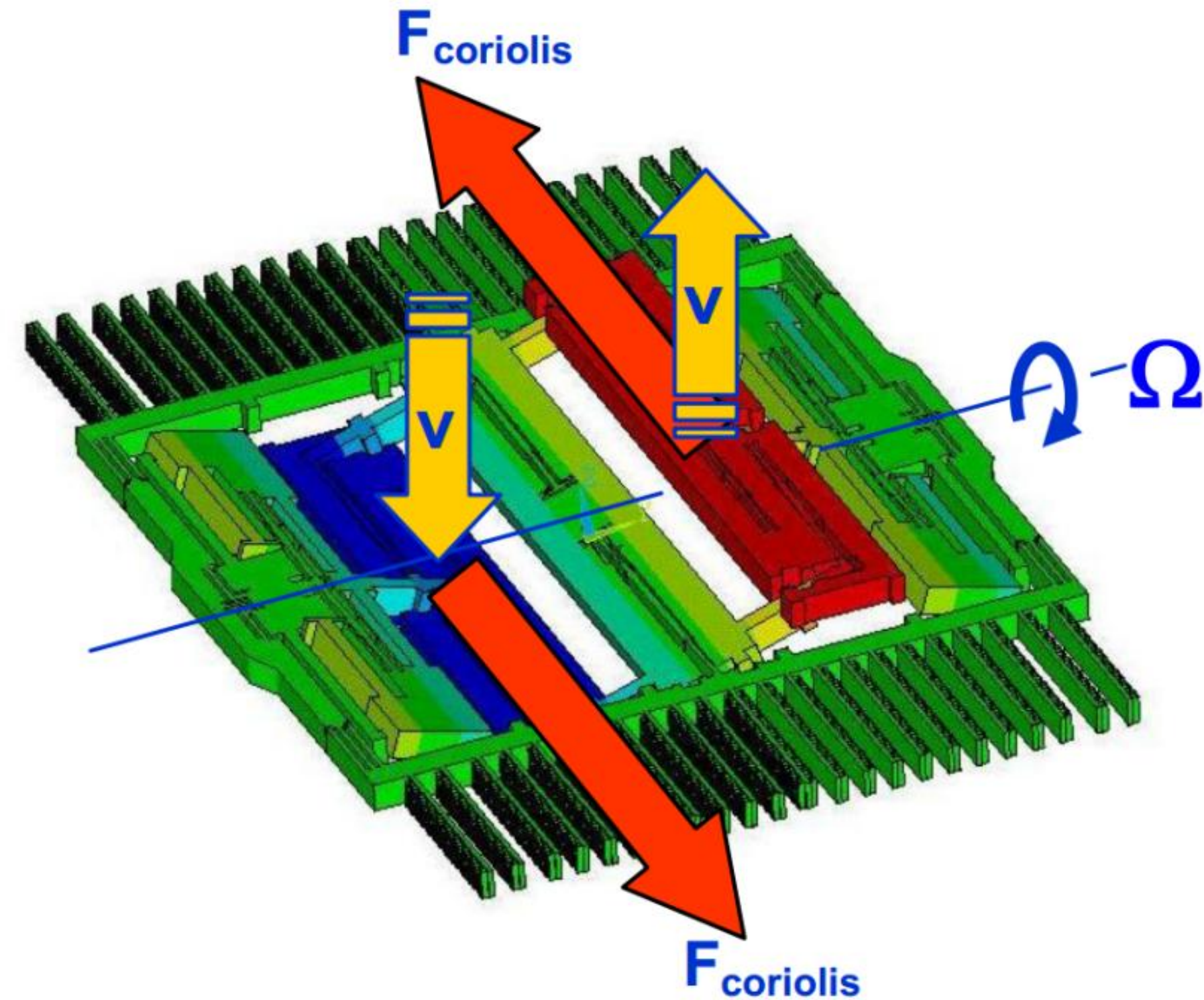
{dasonov,ragrawal}@us.ibm.com

Accelerometer Eavesdropping



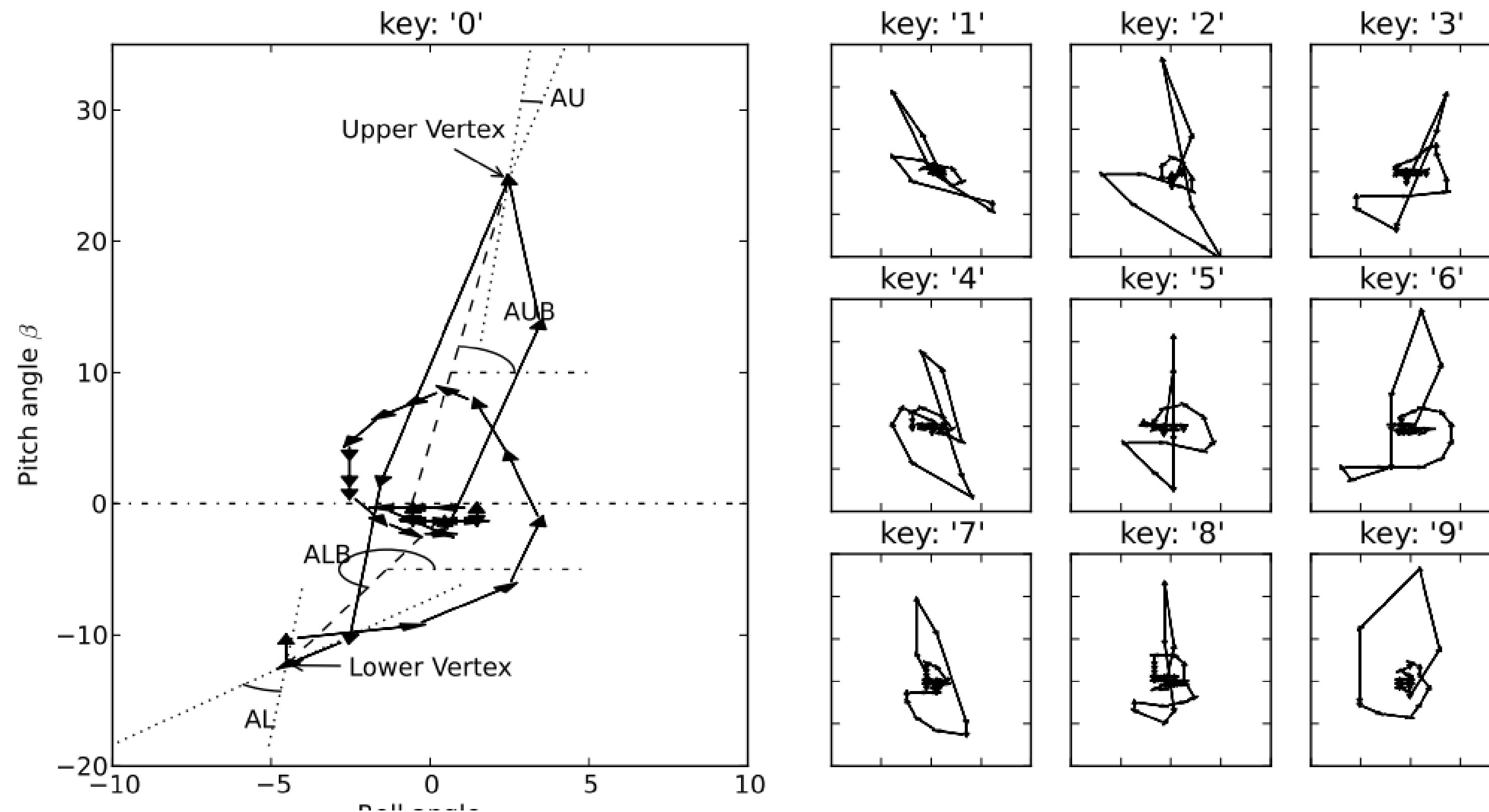
Aviv et al. "Practicality of Accelerometer Side Channels on Smartphones" ACSAC 2012

Gyroscope Eavesdropping



Michalevsky et al. "Gyrophone: Recognizing Speech from Gyroscope Signals" USENIX Security 2014

More Gyroscope



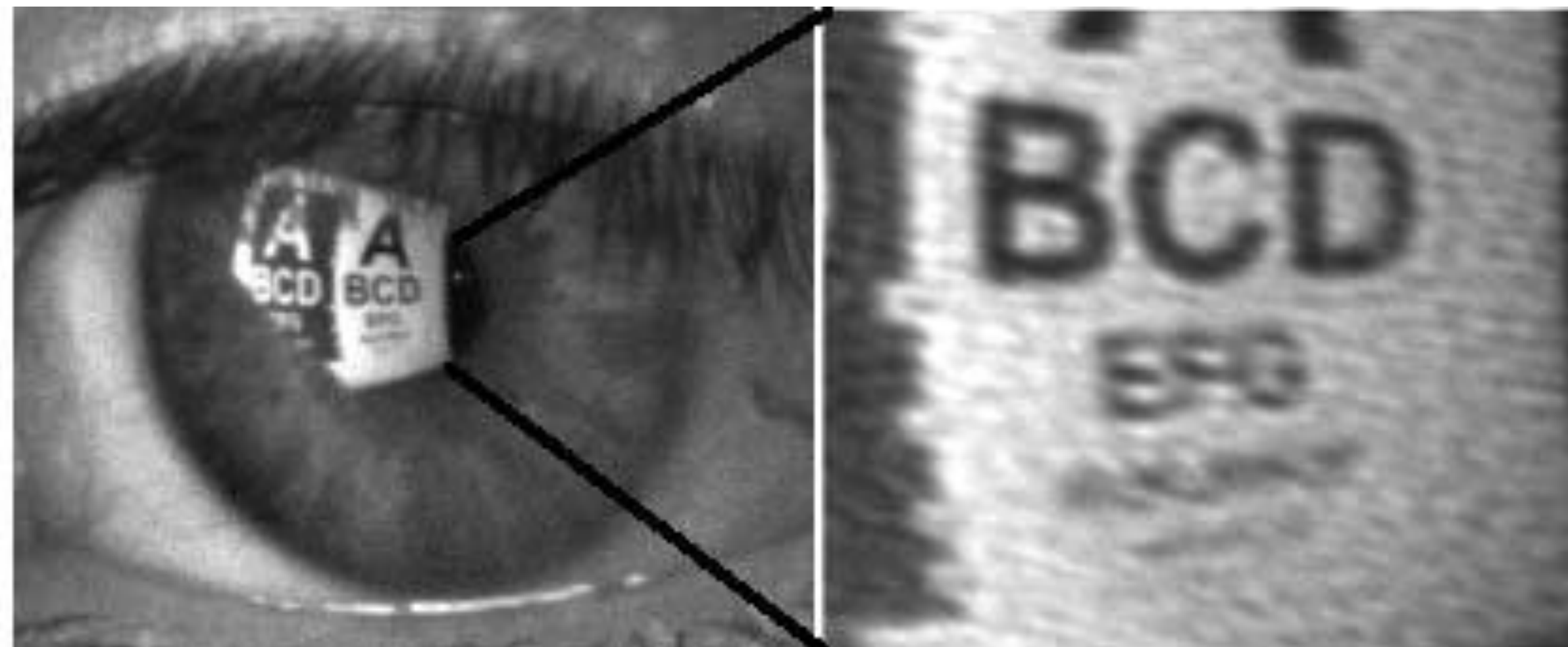
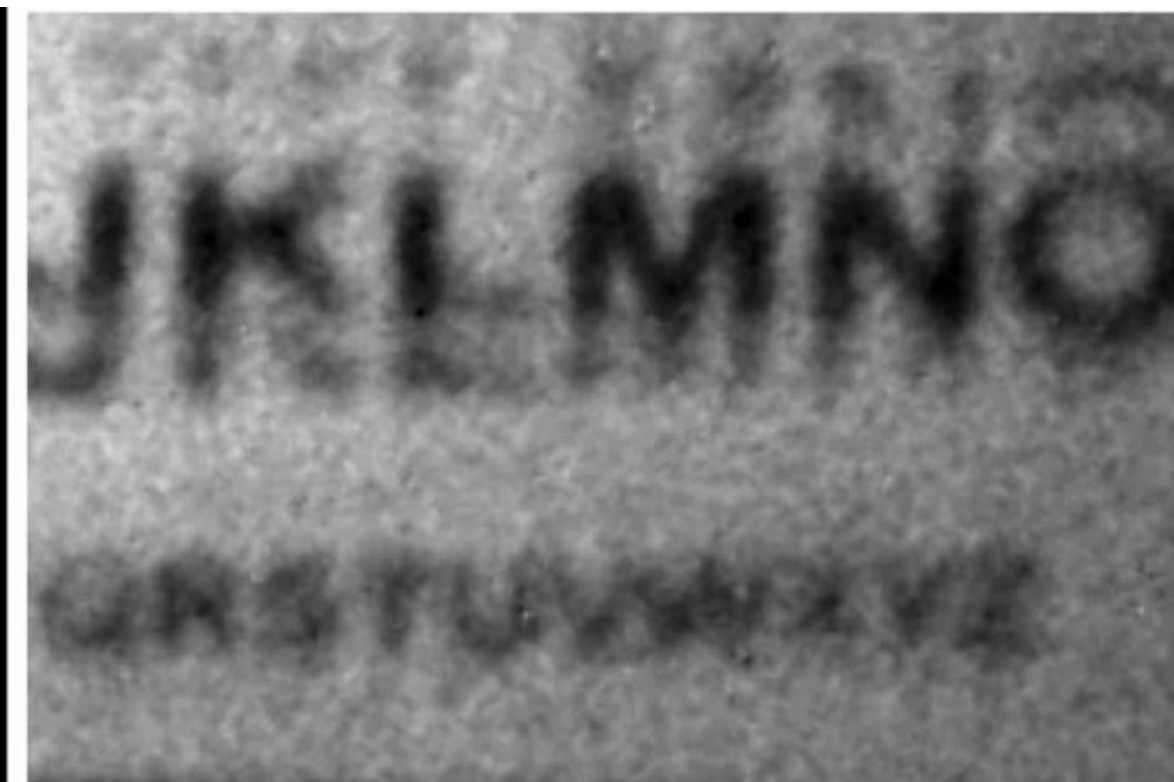
Chen et al. “TouchLogger: Inferring Keystrokes On Touch Screen From Smartphone Motion” HotSec 2011

Keyboard Eavesdropping



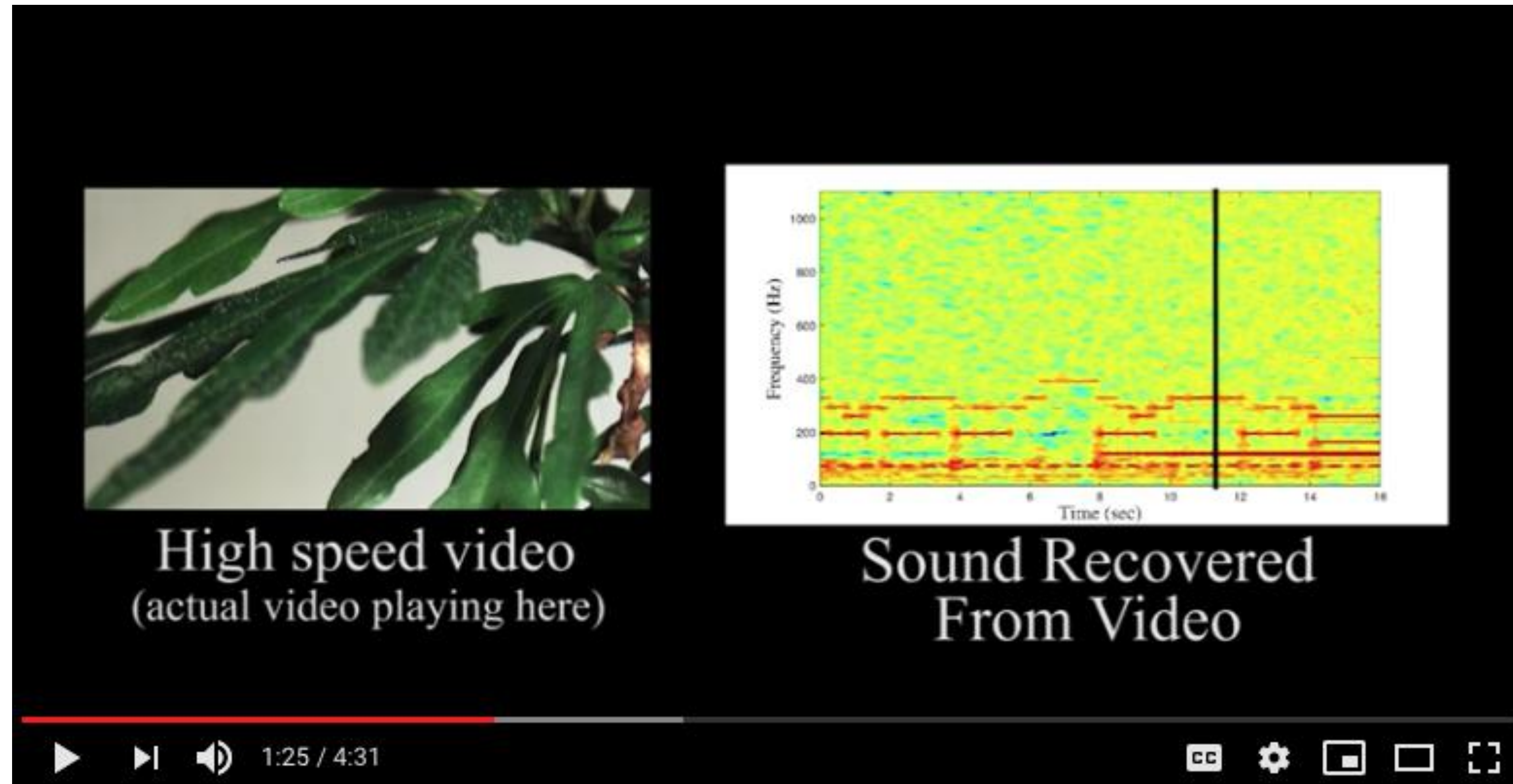
Zhuang et al. “Keyboard Acoustic Emanations Revisited” CCS 2005
Vuagnoux et al. “Compromising Electromagnetic Emanations of Wired and Wireless Keyboards” USENIX Security 2009

Compromising Reflections



Audio from Video

<https://www.youtube.com/watch?v=FKXOucXB4a8>



Davis et al. "The Visual Microphone: Passive Recovery of Sound from Video" SIGGRAPH
2014

Identifying Web Pages: Traffic Analysis

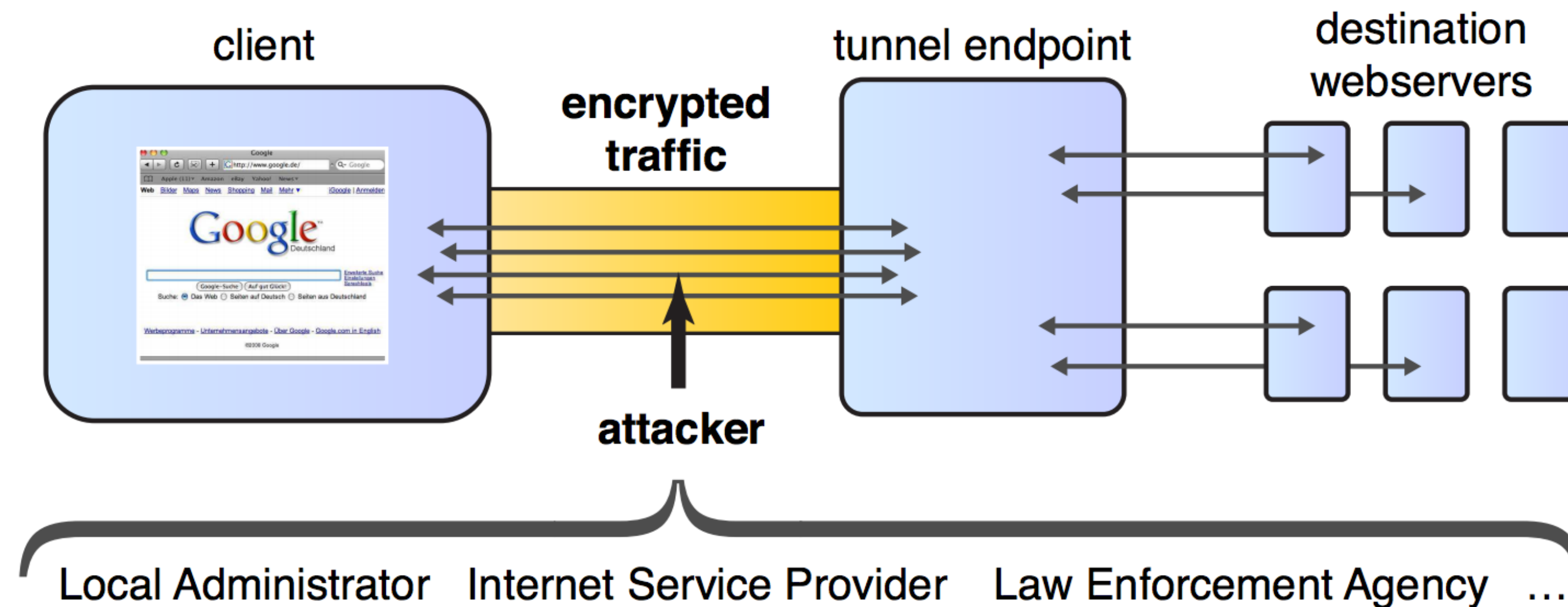


Figure 1: Website fingerprinting scenario and conceivable attackers

Herrmann et al. "Website Fingerprinting: Attacking Popular Privacy Enhancing Technologies with the Multinomial Naïve-Bayes Classifier" CCSW 2009

Identifying Web Pages: Electrical Outlets

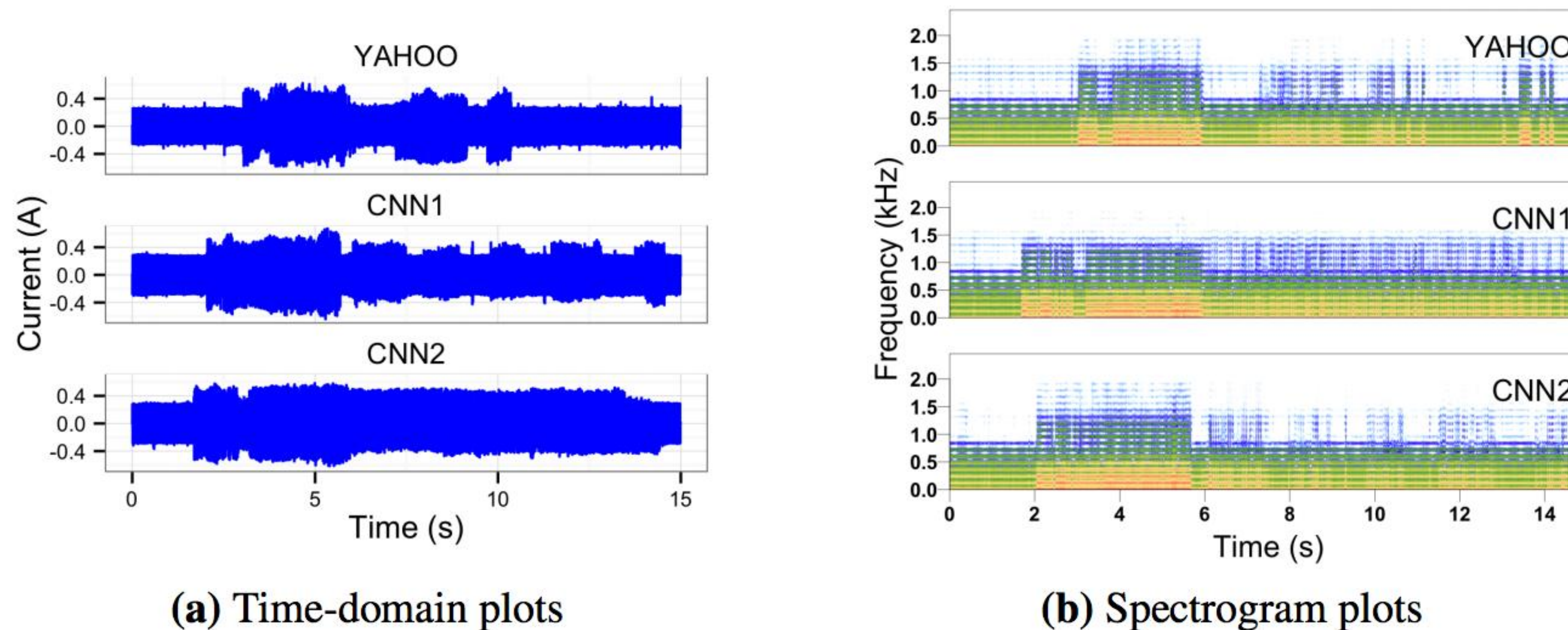


Fig. 1: Time- and frequency-domain plots of several power traces as a MacBook loads two different pages. In the frequency domain, brighter colors represent more energy at a given frequency. Despite the lack of obviously characteristic information in the time domain, the classifier correctly identifies all of the above traces.

Powerline Eavesdropping

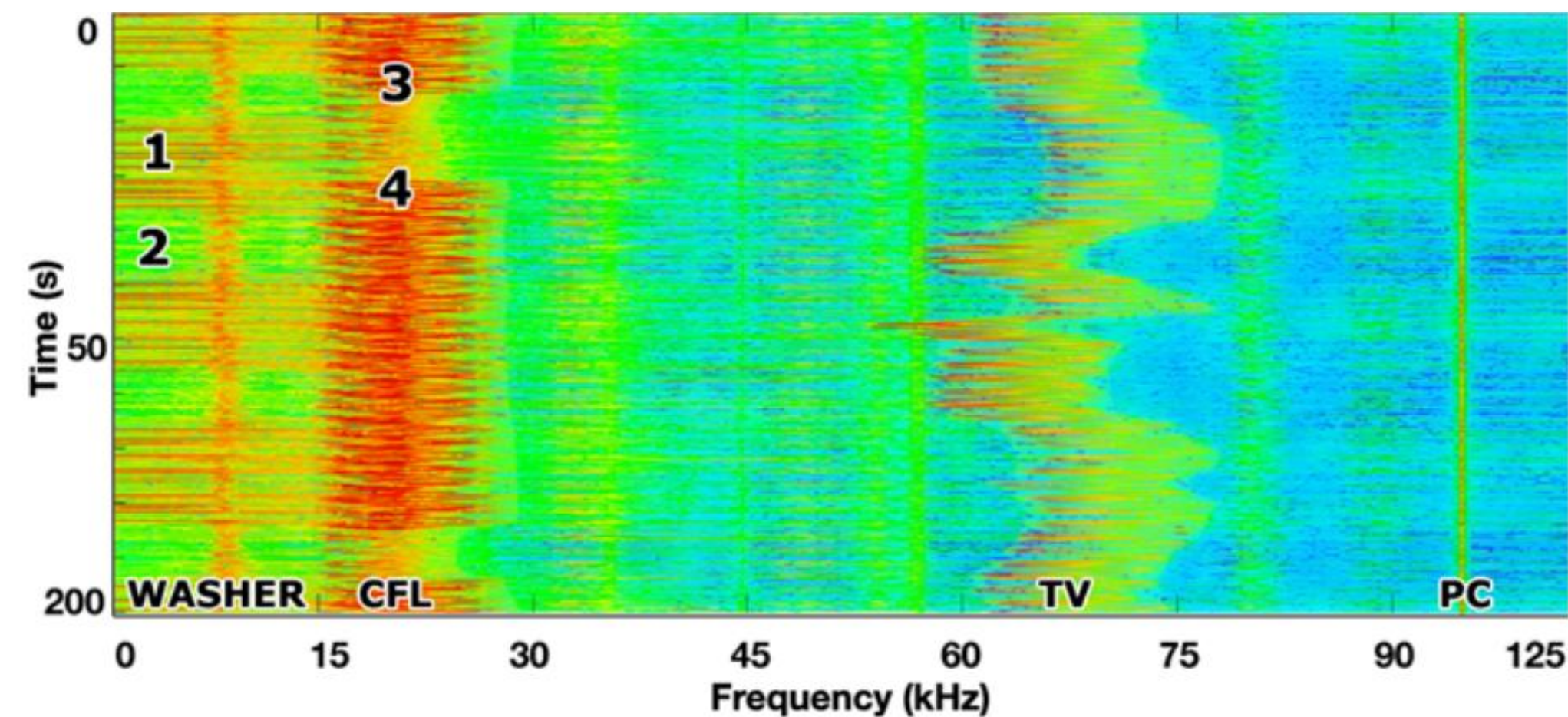
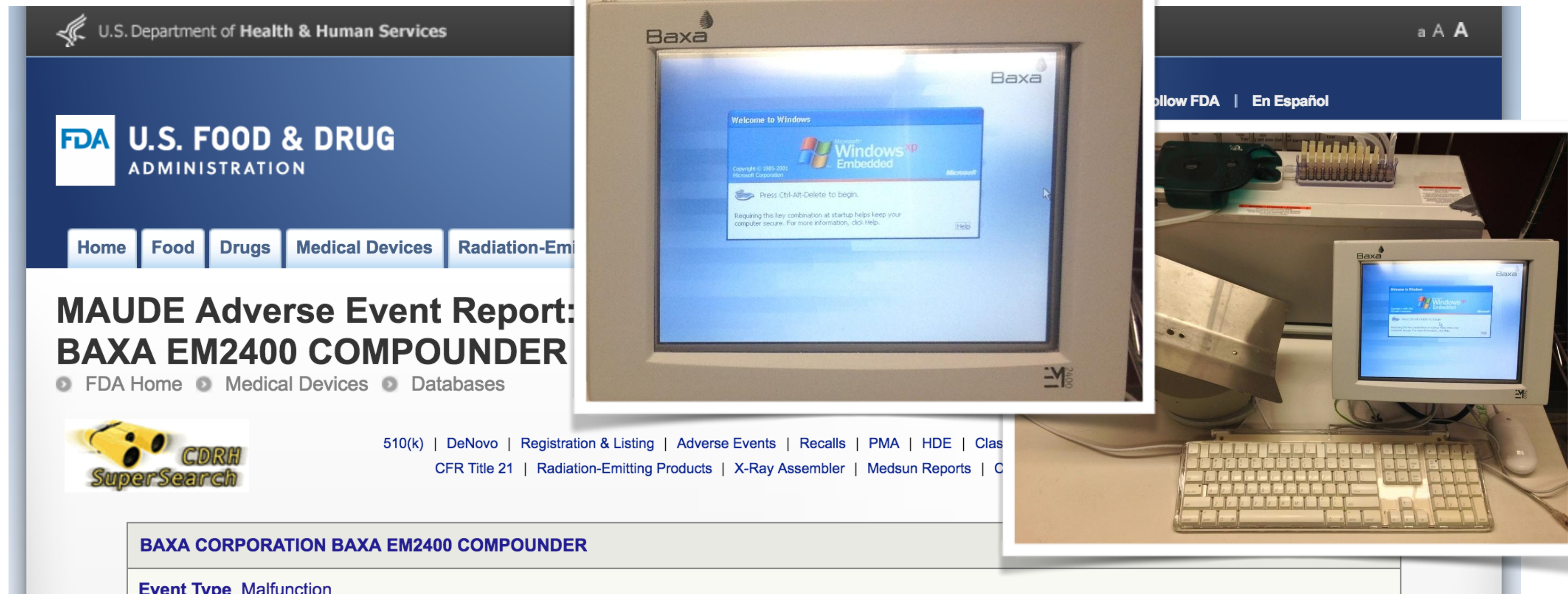


Figure 1: Frequency spectrogram showing various electrical appliances in the home. Washer cycle on (1) and off (2). CFL lamp turning off briefly (3) and then on (4). Note that the TV's (Sharp 42" LCD) EMI shifts in frequency, which happens as screen content changes.

Can Side Channels be beneficial?

Yes! Anomaly Detection



“Recently, the compounder was infected with a virus. It is unknown what effect this virus should have on the operating of the software.”

The Insight

- Embedded medical devices have **specific, limited tasks**
- If we can characterize those tasks, we can **identify later deviations**

Solution Requirement

	No software changes	No updates	No manual configuration	No network connection
Antivirus	✗	✗	✓	✓
Firewall	✓	✓	✗	✗
NIDS	✓	✗	✗	✗
Side Channel	✓	✓	✓	✓

XZ Utils backdoor (CVE-2024-3094)

- CVE stands for Common Vulnerabilities and Exposures
- Widely considered one of the most sophisticated, patient, and terrifying supply-chain attacks in computer history.
- Luckily, it was caught because of a side-channel leak.

The attacker spent over two years using fake accounts and social engineering to slowly gain the trust of a burned-out open-source maintainer.

Eventually, they got maintainer rights and quietly planted a backdoor into liblzma (a compression library used by XZ Utils).

This compression library gets loaded into memory by OpenSSH (the software used to securely log into servers remotely)

XZ Utils backdoor (CVE-2024-3094)

- A Microsoft engineer noticed two strange anomalies when trying to log in via SSH:
 1. SSH logins were generating an unusually high amount of CPU usage.
 2. The login process took about 500 milliseconds longer than usual.
- The delay was a massive red flag!
- The engineer blew the whistle days before the malicious code was scheduled to be integrated into stable, mainstream Linux distributions.